

Package ‘dsClinicalDataHarmonizationClient’

September 15, 2026

Title dsClinicalDataHarmonization client side base functions

Version 0.0.1

Date 2026-09-14

Author Arianna Fontanesi, Simone Avesani, Gospel Ozioma Nnadi

Maintainer Arianna Fontanesi <arianna.fontanesi@univr.it>

Description dsClinicalDataHarmonizationClient -
Federated MDRRA Clinical Data Harmonization and data cleaning on DataSHIELD.

Options datashield.privacyLevel=5

Depends R (>= 3.5)

Encoding UTF-8

LazyData true

License GPL-3

SystemRequirements GNU Fortran

Config/roxygen2/version 8.1.0

RoxygenNote 7.3.2

R topics documented:

ds.cast_NA	2
ds.cast_to_factor	2
ds.check_missing_data	3
ds.check_patient_visit	5
ds.check_residual_na	5
ds.check_types	6
ds.check_type_conversion	6
ds.check_variables	7
ds.compute_derived_variables	8
ds.detect_zero_anomalies	8
ds.dummy_encode_ordinal	9
ds.exploratory_analysis	10
ds.fill_missing_data	11
ds.force_convert_types	12
ds.global_imputation	12
ds.harmonization_clean	13
ds.harmonization_diagnose	15

ds.harmonization_explore	16
ds.harmonization_orchestrator	17
ds.harmonization_report	18
ds.impute_missing_data	19
ds.post_imputation_cleanup	20
ds.remove_columns	21

Index	22
--------------	-----------

ds.cast_NA	<i>Cast "NA" string into NA</i>
------------	---------------------------------

Description

Converts "NA" strings into proper NA values across a server-side data frame, and reports whether any replacements occurred. Calls the disclosure-safe server function `cast_NADS` via `datashield.aggregate` – the transformed data frame is created server-side as a side effect (never returned to the client); only the changed flag comes back.

Server function called: `cast_NADS`

Usage

```
ds.cast_NA(df, modified_obj = NULL, datasources = NULL)
```

Arguments

<code>df</code>	A character string specifying the name of the server-side data frame.
<code>modified_obj</code>	A character string specifying the name of the new server-side object to store the modified data frame. If <code>NULL</code> , a name is generated automatically by appending <code>"_cast_NA"</code> to <code>df</code> .
<code>datasources</code>	A list of DSConnection-class objects obtained after login. If the <code>datasources</code> argument is not specified the default set of connections will be used: see datashield.connections_c

Value

Invisibly returns the name of the newly created server-side object.

ds.cast_to_factor	<i>Cast specified columns to factor on a server-side data frame</i>
-------------------	---

Description

Convert one or more columns of a server-side data frame to factor, via the disclosure-safe server function `cast_to_factorDS`, called through `datashield.aggregate` – the modified data frame is created server-side as a side effect; only column names come back.

Server function called: `cast_to_factorDS`

Usage

```
ds.cast_to_factor(df, columns, modified_obj = NULL, datasources = NULL)
```

Arguments

df	A character string specifying the name of the server-side data frame containing the columns to be converted.
columns	A character vector specifying the names of the columns to convert to factor.
modified_obj	A character string specifying the name of the new server-side object to store the modified data frame. If NULL, a name is generated automatically by appending "_modified" to df.
datasources	A list of DSConnection-class objects obtained after login. If the datasources argument is not specified the default set of connections will be used: see datashield.connections_c

Value

Invisibly returns the name of the newly created server-side object.

ds.check_missing_data *Filter columns of a server-side object based on a missingness threshold*

Description

Computes the percentage of missing values for each variable in a distributed data set and optionally removes variables exceeding a user-defined threshold.

The function supports two filtering strategies:

- "combined": computes a global missingness percentage for each variable as a weighted average across all servers. The same set of variables is removed from all servers.
- "split": evaluates missingness independently on each server and removes server-specific sets of variables.

A missingness table is always displayed. When variables exceed the threshold, they are reported along with their missingness percentages.

If no variables exceed the threshold, no filtering is applied and the original data remain unchanged.

In addition to being displayed, the underlying missingness table(s) are attached as attributes to the returned value, so they can be retrieved for further use (e.g. plotting) without re-parsing the printed messages. Only the attribute relevant to the chosen type is populated; the other is NULL:

- "missing_aggregate": populated only when type = "combined". A single data.frame with variables as row names, one column per server, and a global column with the weighted-average missingness percentage.
- "missing_server": populated only when type = "split". A named list of data.frames (one per server), each with variables as row names and a single column containing that server's missingness percentages.

Server-side functions called: check_missing_dataDS, remove_columnsDS, drop_rows_missingDS – all called via datashield.aggregate; server-side objects are created via each function's internal disclosure-safe assign(..., envir = parent.frame()), never via datashield.assign.expr.

Usage

```
ds.check_missing_data(
  df,
  threshold,
  type = c("combined", "split"),
  remove = c(TRUE, FALSE),
  action = c("drop_column", "drop_rows"),
  preproc_newobj = NULL,
  datasources = NULL
)
```

Arguments

df	A character string specifying the name of the input data frame stored on each server.
threshold	A numeric value between 0 and 100 representing the maximum allowed percentage of missing values per variable.
type	A character string specifying the filtering strategy: "combined" (default) or "split".
remove	A logical value indicating whether flagged variables should be acted on at all. If TRUE (default), the strategy in action is applied. If FALSE, variables exceeding the threshold are only identified and returned, without modifying any server-side object.
action	A character string, one of "drop_column" (default – the flagged VARIABLE is removed for every row, via remove_columnsDS; original behaviour) or "drop_rows" (the variable is kept, and instead only the ROWS missing on that variable are removed, via drop_rows_missingDS). Use "drop_rows" when a variable is too important to drop outright but its missingness is concentrated in a removable subset of rows; use "drop_column" when the variable itself is unusable across most of the cohort. Ignored if remove = FALSE.
preproc_newobj	A character string specifying the name of the output server-side object. If NULL, a name is automatically generated by appending "_preprocessed" to df. Ignored if remove = FALSE.
datasources	A list of DSConnection-class objects. If not provided, the default active connections are used.

Value

- If remove = TRUE: invisibly returns a character string with the name of the created server-side object containing the filtered data. If no variables exceed the threshold, the original data are left unchanged and the intended output object name is still returned.
- If remove = FALSE:
 - For type = "combined": a character vector of variables exceeding the threshold.
 - For type = "split": a named list of character vectors (one per server) containing variables exceeding the threshold.
 - NULL if no variables exceed the threshold.

In every case above, the returned value additionally carries two attributes, "missing_aggregate" and "missing_server", as described in the Description section, which can be retrieved via attr() (e.g. attr(result, "missing_aggregate")).

ds.check_patient_visit

Check patient-visit combinations on a server-side data frame

Description

Reports whether duplicate (patient, visit) combinations exist, using the disclosure-safe check_patient_visitDS server-side function. Only aggregate counts are ever returned or printed here – an earlier version of this function printed the actual duplicated patient IDs to the console (via the server returning the full per-patient-visit table), which was itself a disclosure issue independent of how it was called; that table is no longer computed client-side at all.

Server function called: check_patient_visitDS

Usage

```
ds.check_patient_visit(
  df,
  pat_id_col = "pat_ID",
  visit_col = "Visit",
  datasources = NULL
)
```

Arguments

df	A character string specifying the name of the server-side data frame containing at least the patient identifier and visit columns.
pat_id_col	Character. Name of the patient identifier column. Default "pat_ID".
visit_col	Character. Name of the visit column. Default "Visit_months_from_diagnosis".
datasources	A list of DSConnection-class objects obtained after login. If the datasources argument is not specified, the default set of connections will be used: see datashield.connections_

Value

Invisible per-server list of counts (n_unique_patients, n_unique_pairs, n_duplicate_pairs, max_occurrences). Messages report the same, per server.

ds.check_residual_na *Check for residual missing values after imputation*

Description

Reports, per server, which columns (if any) still contain NA after an imputation step, and how many. Call this BEFORE deciding whether to use ds.post_imputation_cleanup().

Server function called: check_residual_naDS

Usage

```
ds.check_residual_na(df, datasources = NULL)
```

Arguments

`df` Character string naming the (imputed) server-side data frame.

`datasources` A list of [DSConnection-class](#) objects.

Value

Invisible per-server list of named NA counts.

<code>ds.check_types</code>	<i>Check variable types on a server-side data frame</i>
-----------------------------	---

Description

Verify that numeric and categorical variables in a server-side data frame comply with predefined valid ranges and values. Uses the server-side functions `check_numericDS` and `check_categoricalDS` to identify non-compliant columns. Messages are printed for each server indicating which columns, if any, are invalid.

Server functions called: `check_numericDS`, `check_categoricalDS`

Usage

```
ds.check_types(df, datasources = NULL)
```

Arguments

`df` A character string specifying the name of the server-side data frame containing numeric and categorical variables to be checked.

`datasources` A list of [DSConnection-class](#) objects obtained after login. If the `datasources` argument is not specified, the default set of connections will be used: see [datashield.connections_](#)

Value

Invisible NULL. Messages are printed to indicate which columns, if any, are non-compliant on each server.

<code>ds.check_type_conversion</code>	<i>Check whether columns would change type if conversion were forced</i>
---------------------------------------	--

Description

Dry-run report using `detect_type_mismatchesDS` – use this after `ds.check_types()` flags a column, to see whether it's a simple representation issue (e.g. numeric stored as character) that `ds.force_convert_types()` could resolve, before actually forcing it. Never creates a server-side object; already a plain, disclosure-safe aggregate call.

Server function called: `detect_type_mismatchesDS`

Usage

```
ds.check_type_conversion(
  df,
  numeric_cols = NULL,
  categorical_cols = NULL,
  datasources = NULL
)
```

Arguments

df Character string naming the server-side data frame.

numeric_cols Character vector of columns expected to be numeric.

categorical_cols Character vector of columns expected to be a factor.

datasources A list of [DSConnection-class](#) objects.

Value

Invisible list of per-server results.

ds.check_variables	<i>Check variables on a server-side data frame</i>
--------------------	--

Description

Verify that a server-side data frame contains every required variable, report which declared optional variables are present, and flag anything unexpected. Uses the server-side function `check_variablesDS`. Messages are printed for each server indicating discrepancies.

If `variables` is not supplied, this defaults to the RA harmonization protocol's canonical required variable list (fetched server-side from `clinical_variable_dictionaryDS()` on the first connected server, so the client never has to hardcode it), and likewise for optional.

Server function called: `check_variablesDS`

Usage

```
ds.check_variables(df, variables = NULL, optional = NULL, datasources = NULL)
```

Arguments

df A character string specifying the name of the server-side data frame to check.

variables A character vector specifying the expected REQUIRED variable names. NULL (default) uses the package's canonical required list.

optional A character vector specifying OPTIONAL variable names – these are never reported as "missing", only as present/absent for informational purposes, and are excluded from the "extra" list. NULL (default) uses the package's canonical optional list.

datasources A list of [DSConnection-class](#) objects obtained after login. If the `datasources` argument is not specified, the default set of connections will be used: see [datashield.connections_](#)

Value

Invisible list of results returned by the server-side function. Each element corresponds to a server and contains missing, optional_present, and extra.

ds.compute_derived_variables

Compute derived clinical variables on a server-side data frame

Description

Adds USPDGS (= USPD + USGS) and/or DAS2C (= $\sqrt{\text{SJC28}} + 0.6 \cdot \log_{10}(\text{CRP})$) wherever the required source columns are available at a given site, via the disclosure-safe server function compute_derived_variablesDS called through datashield.aggregate – the modified data frame is created server-side as a side effect; only column names come back.

Server function called: compute_derived_variablesDS

Usage

```
ds.compute_derived_variables(df, newobj = NULL, datasources = NULL)
```

Arguments

df	A character string specifying the name of the server-side data frame.
newobj	A character string specifying the name of the new server-side object. If NULL, a name is generated by appending "_derived" to df.
datasources	A list of DSConnection-class objects. If not specified, the default set of connections is used.

Value

Invisibly returns the name of the newly created server-side object.

ds.detect_zero_anomalies

Detect suspicious zero-inflation (possible erroneous numeric cast)

Description

Flags numeric columns where zeros look like a cast artefact rather than genuine clinical zeros – see detect_zero_anomaliesDS for the two heuristics used. Run this after any type coercion step (ds.force_convert_types, ds.cast_NA, imputation) and, generally, as part of routine data-quality review.

Server function called: detect_zero_anomaliesDS

Usage

```
ds.detect_zero_anomalies(
  df,
  cols = NULL,
  zero_prop_threshold = 0.3,
  spike_ratio_threshold = 3,
  num_bins = 10,
  nfilter = 5,
  datasources = NULL
)
```

Arguments

df Character string naming the server-side data frame.

cols Character vector restricting the check to specific columns. NULL (default): every numeric column.

zero_prop_threshold, spike_ratio_threshold, num_bins, nfilter See `detect_zero_anomaliesDS`.

datasources A list of [DSConnection-class](#) objects.

Value

Invisible per-server list of per-column zero-anomaly stats.

`ds.dummy_encode_ordinal`

Dummy-encode ordinal DMARD-style variables on a server-side data frame

Description

Adds integer 0/1 dummy columns for the specified ordinal/nominal variables (defaults to the DMARD family) while keeping the original column, itself coerced to integer. Uses the disclosure-safe server function `dummy_encode_ordinalDS`, called through `datashield.aggregate` – the modified data frame is created server-side as a side effect; only column names come back. `IP_score` and `EP_score` are always excluded server-side even if included in columns, since those are severity annotations, not treatment categories.

Server function called: `dummy_encode_ordinalDS`

Usage

```
ds.dummy_encode_ordinal(
  df,
  columns = c("csDMARD1", "csDMARD2", "csDMARD3", "bDMARD", "tsDMARD", "GC_type"),
  newobj = NULL,
  datasources = NULL
)
```

Arguments

df	A character string specifying the name of the server-side data frame.
columns	A character vector of columns to dummy-encode. Defaults to c("csDMARD1", "csDMARD2", "csDMARD3", "bDMARD", "tsDMARD", "GC_type").
newobj	A character string specifying the name of the new server-side object. If NULL, a name is generated by appending "_dummy" to df.
datasources	A list of DSConnection-class objects. If not specified, the default set of connections is used.

Value

Invisibly returns the name of the newly created server-side object.

ds.exploratory_analysis

Non-disclosive exploratory analysis of a server-side data frame

Description

Per-site and aggregated descriptive statistics, distributions, category frequencies, correlations, and group-wise trends, using the server function `exploratory_analysisDS`. Only aggregate statistics ever return – see `exploratory_analysisDS` for the exact disclosure-control convention (nfilter-suppressed cells/bins).

Usually called via `ds.harmonization_explore()` rather than directly – that wrapper defaults to the cleaned dataset and builds a full figure set. Use this function directly for finer control over which columns/grouping to summarize.

Server function called: `exploratory_analysisDS`

Usage

```
ds.exploratory_analysis(
  df,
  numeric_cols = NULL,
  categorical_cols = NULL,
  group_col = NULL,
  num_bins = 20,
  nfilter = 5,
  datasources = NULL
)
```

Arguments

df	Character string naming the server-side data frame.
numeric_cols, categorical_cols	Character vectors restricting which columns to summarize. NULL (default): every numeric / every non-numeric column respectively.
group_col	Optional column for group-wise numeric means (e.g. a visit number, for a longitudinal trend).

num_bins	Number of histogram bins per numeric column. Default 20.
nfilter	Disclosure/stability floor.
datasources	A list of DSConnection-class objects.

Value

Invisible per-server list, the raw exploratory_analysisDS output.

ds.fill_missing_data *Fill missing visit data on a server-side data frame*

Description

Fills missing values in a specified column within each patient, ordered by visit time from diagnosis, via the disclosure-safe server function `fill_missing_dataDS`, called through `datashield.aggregate` – the modified data frame is created server-side as a side effect; only row/fill counts come back.

Server function called: `fill_missing_dataDS`

Usage

```
ds.fill_missing_data(
  df,
  pat_id_col,
  visit_col,
  value_col,
  filled_newobj = NULL,
  datasources = NULL
)
```

Arguments

df	A character string specifying the name of the server-side data frame containing at least the patient identifier column, the visit time column, and the variable to be forward-filled.
pat_id_col	A character string specifying the name of the patient identifier column.
visit_col	A character string specifying the name of the visit time column.
value_col	A character string specifying the name of the column whose missing values should be forward-filled.
filled_newobj	A character string specifying the name of the new server-side object that will store the modified data frame. If not provided, the default name is <code>paste0(df, "_filled")</code> .
datasources	A list of DSConnection-class objects obtained after login. If not specified, the default set of connections is used: see datashield.connections_default .

Value

Invisibly returns the name of the newly created server-side object.

`ds.force_convert_types`
Force type conversion on a server-side data frame

Description

Actually performs the conversion identified by `ds.check_type_conversion()`. This is an explicit, opt-in step – the harmonization pipeline never silently coerces types on its own. Calls the disclosure-safe server function `force_convert_typesDS` via `datashield.aggregate` – the converted data frame is created server-side as a side effect; only column names and non-disclosive zero-anomaly diagnostics for the newly-converted numeric columns come back (see `detect_zero_anomaliesDS`).

Server function called: `force_convert_typesDS`

Usage

```
ds.force_convert_types(
  df,
  numeric_cols = NULL,
  categorical_cols = NULL,
  newobj = NULL,
  datasources = NULL
)
```

Arguments

<code>df</code>	Character string naming the server-side data frame.
<code>numeric_cols</code>	Character vector of columns to force to numeric.
<code>categorical_cols</code>	Character vector of columns to force to factor.
<code>newobj</code>	Name of the new server-side object. Defaults to <code>paste0(df, "_typed")</code> .
<code>datasources</code>	A list of DSConnection-class objects.

Value

Invisibly returns `newobj`.

`ds.global_imputation` *Global imputation of a server-side object using MICE*

Description

Performs pooled (not patient-grouped) imputation via the disclosure-safe server function `global_imputationDS`, called through `datashield.aggregate` – the imputed data frame is created server-side as a side effect; only row/column counts and the per-column imputation method names (never data) come back.

Server function called: `global_imputationDS`

Usage

```
ds.global_imputation(
  df,
  id_col = NULL,
  m = 5,
  maxit = 30,
  imputed_newobj = NULL,
  datasources = NULL
)
```

Arguments

df	A character string specifying the name of the server-side data frame on which the global imputation is performed.
id_col	A character string indicating the name of the identifier column that is excluded from imputation.
m	A numeric value representing the number of imputations to generate. Default is 5.
maxit	A numeric value representing the maximum number of MICE iterations per imputation chain. Default is 30.
imputed_newobj	A character string specifying the name of the new imputed object to be created on each server. If NULL, a name is generated automatically by appending "_imputed" to df.
datasources	A list of DataSHIELD connections. If NULL, the active connections returned by <code>datashield.connections_find()</code> are used.

Value

Invisibly returns the name of the newly created server-side object.

ds.harmonization_clean

Harmonization data cleaning (standalone)

Description

Runs the data-cleaning pipeline in this fixed order (each step is one of this package's existing functions; a step is skipped, not silently altered, if its argument is NULL/FALSE):

1. `ds.cast_NA()` – normalize literal "NA" strings.
2. `ds.check_missing_data(..., action = missing_action)` – drop columns or rows exceeding `missing_threshold`.
3. `ds.force_convert_types()` (only if `force_numeric_cols` or `force_categorical_cols` supplied).
4. `ds.compute_derived_variables()` (USPDGS, DAS2C).
5. `ds.dummy_encode_ordinal()` (DMARD family by default).
6. `ds.impute_missing_data()` (longitudinal, default) or `ds.global_imputation()`, or skipped if `imputation_method = "none"`.

7. `ds.post_imputation_cleanup()` (only if `post_imputation_drop = TRUE`).

The final result is assigned, on every server, to `final_newobj` (default `"Draw_cleaned"`). If `run_post_diagnosis = TRUE` (default), `ds.harmonization_diagnose()` is then run on the cleaned object and its report/figures are included in the return value – set this to `FALSE` when the caller (e.g. `ds.harmonization_orchestrator(mode = "diagnoseclean")`) is going to build its own before/after comparison instead.

Usage

```
ds.harmonization_clean(
  df,
  missing_threshold = 20,
  missing_type = c("combined", "split"),
  missing_action = c("drop_column", "drop_rows"),
  dummy_cols = c("csDMARD1", "csDMARD2", "csDMARD3", "bDMARD", "tsDMARD", "GC_type"),
  compute_derived = TRUE,
  force_numeric_cols = NULL,
  force_categorical_cols = NULL,
  imputation_method = c("longitudinal", "global", "none"),
  post_imputation_drop = FALSE,
  pat_id_col = "pat_ID",
  visit_col = "Visit",
  nfilter = 5,
  final_newobj = "Draw_cleaned",
  run_post_diagnosis = TRUE,
  required_vars = NULL,
  optional_vars = NULL,
  zero_prop_threshold = 0.3,
  spike_ratio_threshold = 3,
  datasources = NULL
)
```

Arguments

<code>df</code>	Character string naming the raw server-side input data frame.
<code>missing_threshold</code>	Numeric, % missing above which a variable is flagged.
<code>missing_type</code>	"combined" or "split" – see <code>ds.check_missing_data</code> .
<code>missing_action</code>	"drop_column" or "drop_rows".
<code>dummy_cols</code>	Columns to ordinal-dummy-encode. NULL to skip.
<code>compute_derived</code>	Logical, run <code>ds.compute_derived_variables()</code> .
<code>force_numeric_cols</code> , <code>force_categorical_cols</code>	Columns to force-convert. NULL (default): step skipped.
<code>imputation_method</code>	"longitudinal" (default), "global", or "none".
<code>post_imputation_drop</code>	Logical, drop rows still incomplete after imputation.
<code>pat_id_col</code> , <code>visit_col</code>	Used for longitudinal imputation grouping/ordering, and (if <code>run_post_diagnosis</code>) the post-cleaning duplicate check.

nfilter	Disclosure/stability floor used throughout.
final_newobj	Name of the final cleaned object. Default "Draw_cleaned".
run_post_diagnosis	Logical, run ds.harmonization_diagnose() on the cleaned result and include its report/figures. Default TRUE.
required_vars, optional_vars, zero_prop_threshold, spike_ratio_threshold	Passed through to the post-cleaning diagnosis, if run.
datasources	A list of DSConnection-class objects.

Value

A list:

- final_object: final_newobj.
- cleaning_log: named list, one entry per pipeline step actually run.
- diagnosis, report, figures: present only if run_post_diagnosis = TRUE (else NULL).

ds.harmonization_diagnose

Harmonization diagnosis (standalone)

Description

Runs the non-disclosive harmonization diagnosis (harmonization_diagnosisDS: variable presence, "NA"-string contamination, numeric/categorical range conformity, missingness %, patient-visit duplicates, zero-inflation anomalies) on a single server-side object, and returns a report + ggplot2 figures. Usable standalone, and used internally by ds.harmonization_clean() (post-cleaning snapshot) and ds.harmonization_orchestrator() (before AND after snapshots).

Usage

```
ds.harmonization_diagnose(
  df,
  required_vars = NULL,
  optional_vars = NULL,
  pat_id_col = "pat_ID",
  visit_col = "Visit",
  zero_prop_threshold = 0.3,
  spike_ratio_threshold = 3,
  nfilter = 5,
  datasources = NULL
)
```

Arguments

df	Character string naming the server-side object to diagnose.
required_vars, optional_vars	Character vectors. NULL (default) fetches the package's canonical dictionary from the first connected server.

pat_id_col, visit_col
 Patient id / visit columns used for the duplicate check.

zero_prop_threshold, spike_ratio_threshold
 See detect_zero_anomaliesDS.

nfilter
 Disclosure/stability floor.

datasources
 A list of [DSConnection-class](#) objects.

Value

A list:

- object: df (echoed back for convenience).
- diagnosis: raw per-server harmonization_diagnosisDS output.
- report: list with missingness, conformity, zero_anomalies, duplicates – each with per-site (and, for missingness/conformity, an aggregated) data.frame.
- figures: named list of ggplot objects, or NULL if ggplot2 isn't installed.

ds.harmonization_explore

Harmonization orchestrator: exploratory analysis

Description

The third orchestrator alongside ds.harmonization_diagnose() and ds.harmonization_clean() – rather than checking conformity or modifying data, this one describes the data: distributions, category frequencies, correlations between numeric variables, and (optionally) a simple longitudinal trend. Defaults to running on "Draw_cleaned", i.e. the object ds.harmonization_clean() / ds.harmonization_orchestrator() produce, since exploring cleaned, harmonized data is almost always what's wanted – pass a different df to explore anything else (e.g. the raw input, for comparison).

Builds on ds.exploratory_analysis() (server function exploratory_analysisDS): every number behind the report/figures is an aggregate statistic, binned histogram count, or correlation cell that already passed the site's nfilter floor – no row-level value is ever returned.

Usage

```
ds.harmonization_explore(
  df = "Draw_cleaned",
  numeric_cols = NULL,
  categorical_cols = NULL,
  group_col = "Visit",
  num_bins = 20,
  nfilter = 5,
  datasources = NULL
)
```

Arguments

df	Character string naming the server-side object to explore. Default "Draw_cleaned".
numeric_cols, categorical_cols	Character vectors restricting which columns to summarize. NULL (default): every numeric / every non-numeric column respectively.
group_col	Optional column for a group-wise trend figure (mean of each numeric variable at each level). Default "Visit"; set NULL to skip.
num_bins	Number of histogram bins per numeric column. Default 20.
nfilter	Disclosure/stability floor.
datasources	A list of DSConnection-class objects.

Value

A list:

- object: df.
- stats: raw per-server exploratory_analysisDS output.
- report: list with numeric_summary (per-site data.frame), categorical_summary (named list of per-site data.frames, one per variable), and correlation (named list of per-site correlation-matrix data.frames, one per server).
- figures: named list of ggplot objects (histograms – one per numeric column per server –, a categorical-frequency bar chart per variable, a correlation heatmap per server, and a group-trend line chart if group_col was usable), or NULL if ggplot2 isn't installed.

ds.harmonization_orchestrator

Harmonization orchestrator (diagnose / clean / diagnoseclean)

Description

Thin dispatcher over the two standalone building blocks, ds.harmonization_diagnose() and ds.harmonization_clean(). In every mode, a report (tables) and, if ggplot2 is installed, figures are returned – nothing here ever returns row-level data, only the aggregate statistics each sub-function already produces.

Usage

```
ds.harmonization_orchestrator(
  df,
  mode = c("diagnoseclean", "diagnose", "clean"),
  required_vars = NULL,
  optional_vars = NULL,
  pat_id_col = "pat_ID",
  visit_col = "Visit",
  missing_threshold = 20,
  missing_type = c("combined", "split"),
  missing_action = c("drop_column", "drop_rows"),
  dummy_cols = c("csDMARD1", "csDMARD2", "csDMARD3", "bDMARD", "tsDMARD", "GC_type"),
```

```

compute_derived = TRUE,
force_numeric_cols = NULL,
force_categorical_cols = NULL,
imputation_method = c("longitudinal", "global", "none"),
post_imputation_drop = FALSE,
zero_prop_threshold = 0.3,
spike_ratio_threshold = 3,
nfilter = 5,
final_newobj = "Draw_cleaned",
datasources = NULL
)

```

Arguments

df	Character string naming the raw server-side input data frame.
mode	One of: • "diagnose": runs <code>ds.harmonization_diagnose(df, ...)</code> only. Nothing is modified or assigned server-side. • "clean": runs <code>ds.harmonization_clean(df, ...)</code> only – cleans, assigns <code>final_newobj</code>, and reports/plots the CLEANED data alone (no comparison to the raw input, since it was never diagnosed in this mode). • "diagnoseclean" (default): diagnoses <code>df</code> (before), cleans it into <code>final_newobj</code>, diagnoses that (after), and returns a combined before/after report + figures.
required_vars, optional_vars, pat_id_col, visit_col, zero_prop_threshold, spike_ratio_threshold, nfilter	See <code>ds.harmonization_diagnose()</code> .
missing_threshold, missing_type, missing_action, dummy_cols, compute_derived, force_numeric_cols, force_categorical_cols, imputation_method, post_imputation_drop, final_newobj	See <code>ds.harmonization_clean()</code> . Ignored when <code>mode = "diagnose"</code> .
datasources	A list of DSConnection-class objects.

Value

- `mode = "diagnose"`: the return value of `ds.harmonization_diagnose()` (object, diagnosis, report, figures).
- `mode = "clean"`: the return value of `ds.harmonization_clean()` (final_object, cleaning_log, diagnosis, report, figures).
- `mode = "diagnoseclean"`: a list with `final_object`, `before`, `after` (raw diagnosis snapshots), `cleaning_log`, and a before/after report/figures.

ds.harmonization_report

Non-disclosive harmonization report (tables + figures)

Description

One-call orchestrator that pulls only aggregate, disclosure-checked statistics from every site (via `harmonization_summaryDS`) and turns them into a small set of report tables and `ggplot2` figures the analyst can put directly into a harmonization report. No row-level or patient-level value is ever

transmitted from any site – every number behind these figures is a count, percentage, or summary statistic that already passed the server-side `nfilter` floor.

Figures produced (as a named list of `ggplot` objects, only if `ggplot2` is installed – if not, only the tables are returned):

- `missingness_plot`: bar chart of % missing per variable, faceted by server.
- `numeric_summary_plot`: point-range plot (median, IQR) per numeric variable, faceted by server.
- `categorical_counts_plot`: one bar chart per categorical variable (list element), faceted by server, cells below the site's `nfilter` shown as blank/NA.

Server function called: `harmonization_summaryDS`

Usage

```
ds.harmonization_report(df, nfilter = 5, datasources = NULL)
```

Arguments

- | | |
|--------------------------|--|
| <code>df</code> | Character string naming the server-side data frame to summarize. |
| <code>nfilter</code> | Minimum count required before a statistic/cell is released (passed straight through to <code>harmonization_summaryDS</code>). |
| <code>datasources</code> | A list of DSConnection-class objects. |

Value

A list:

- `missingness_table`: data.frame, variable x server % missing.
- `numeric_summary_table`: data.frame, one row per (server, variable) numeric summary.
- `categorical_summary`: named list (per variable) of data.frames, one row per (server, category, count).
- `figures`: named list of `ggplot` objects (see above), or NULL if `ggplot2` is not available.

`ds.impute_missing_data`

Longitudinal (patient-grouped) MICE imputation – preferred method

Description

Client wrapper for the disclosure-safe server function `longitudinal_imputationDS`, the preferred imputation strategy for this harmonization module: patient-grouped, visit-ordered, predictive mean matching. Called through `datashield.aggregate` – the imputed data frame is created server-side as a side effect; only row/patient counts (never data) come back. Prefer this over `ds.global_imputation()` whenever the data are longitudinal.

Server function called: `longitudinal_imputationDS`

Usage

```
ds.impute_missing_data(
  df,
  pat_id_col = "pat_ID",
  visit_col = "Visit",
  exclude_cols = NULL,
  m = 5,
  maxit = 30,
  nfilter = 5,
  imputed_newobj = NULL,
  datasources = NULL
)
```

Arguments

df	Character string naming the server-side data frame.
pat_id_col	Name of the patient identifier column. Default "pat_ID".
visit_col	Name of the visit-ordering column. Default "Visit".
exclude_cols	Character vector of columns to exclude from the MICE predictor matrix.
m	Number of imputations per patient. Default 5.
maxit	Maximum MICE iterations per chain. Default 30.
nfilter	Minimum site row count required to proceed at all. Default 5.
imputed_newobj	Name of the new server-side object. Defaults to paste0(df, "_imputed").
datasources	A list of DSConnection-class objects.

Value

Invisibly returns imputed_newobj.

ds.post_imputation_cleanup

Drop rows still incomplete after imputation

Description

Explicit, opt-in row-drop for whatever residual NAs remain after imputation. Run `ds.check_residual_na()` first to see the scale of what would be dropped. Calls the disclosure-safe server function `post_imputation_cleanupDS` via `datashield.aggregate` – the cleaned data frame is created server-side as a side effect; only row counts come back.

Server function called: `post_imputation_cleanupDS`

Usage

```
ds.post_imputation_cleanup(df, newobj = NULL, datasources = NULL)
```

Arguments

df	Character string naming the (imputed) server-side data frame.
newobj	Name of the new server-side object. Defaults to <code>paste0(df, "_complete")</code> .
datasources	A list of DSConnection-class objects.

Value

Invisibly returns newobj.

ds.remove_columns	<i>Remove specified columns from a server-side data frame</i>
-------------------	---

Description

Remove one or more columns via the disclosure-safe server function `remove_columnsDS`, called through `datashield.aggregate` – the modified data frame is created server-side as a side effect; only column names/counts come back.

Server function called: `remove_columnsDS`

Usage

```
ds.remove_columns(df, col_names, newobj = NULL, datasources = NULL)
```

Arguments

df	A character string specifying the name of the server-side data frame from which columns will be removed.
col_names	A character vector specifying the names of the columns to remove.
newobj	A character string specifying the name of the new server-side object to store the modified data frame. If <code>NULL</code> , a name is generated automatically by appending <code>"_filtered"</code> to <code>df</code> .
datasources	a list of DSConnection-class objects obtained after login. If the <code>datasources</code> argument is not specified the default set of connections will be used: see datashield.connections_

Value

Invisibly returns the name of the newly created server-side object.

Index

`datashield.connections_default`, [2](#), [3](#),
[5–7](#), [11](#), [21](#)
`ds.cast_NA`, [2](#)
`ds.cast_to_factor`, [2](#)
`ds.check_missing_data`, [3](#)
`ds.check_patient_visit`, [5](#)
`ds.check_residual_na`, [5](#)
`ds.check_type_conversion`, [6](#)
`ds.check_types`, [6](#)
`ds.check_variables`, [7](#)
`ds.compute_derived_variables`, [8](#)
`ds.detect_zero_anomalies`, [8](#)
`ds.dummy_encode_ordinal`, [9](#)
`ds.exploratory_analysis`, [10](#)
`ds.fill_missing_data`, [11](#)
`ds.force_convert_types`, [12](#)
`ds.global_imputation`, [12](#)
`ds.harmonization_clean`, [13](#)
`ds.harmonization_diagnose`, [15](#)
`ds.harmonization_explore`, [16](#)
`ds.harmonization_orchestrator`, [17](#)
`ds.harmonization_report`, [18](#)
`ds.impute_missing_data`, [19](#)
`ds.post_imputation_cleanup`, [20](#)
`ds.remove_columns`, [21](#)